

Homework5: AVL Tree

Implementation

如下实现了几个关键的操作：

1. 使用 `AVLNode` 类定义节点，包含值、左 / 右子树、高度三个属性；
2. 定义左旋、右旋基本操作；
3. 按二叉搜索树规则插入，更新节点高度，计算平衡因子，对**左左**、**右右**、**左右**、**右左**四种失衡情况执行旋转恢复平衡；
4. 先按二叉搜索树删除节点，再更新高度并调整平衡，处理双分支节点时用右子树最小值替换；
5. 中序遍历和层序遍历两个最基本的遍历方式。

详细代码见 [AVL Tree.ipynb](#)

Performance test results and analysis

```
=====
第一步：插入元素，每次插入后输出 中序遍历
```

```
=====
插入第1个元素 50 → 中序遍历: 50
插入第2个元素 30 → 中序遍历: 30 50
插入第3个元素 70 → 中序遍历: 30 50 70
插入第4个元素 20 → 中序遍历: 20 30 50 70
插入第5个元素 40 → 中序遍历: 20 30 40 50 70
插入第6个元素 60 → 中序遍历: 20 30 40 50 60 70
插入第7个元素 80 → 中序遍历: 20 30 40 50 60 70 80
插入第8个元素 10 → 中序遍历: 10 20 30 40 50 60 70 80
插入第9个元素 25 → 中序遍历: 10 20 25 30 40 50 60 70 80
插入第10个元素 35 → 中序遍历: 10 20 25 30 35 40 50 60 70 80
插入第11个元素 45 → 中序遍历: 10 20 25 30 35 40 45 50 60 70 80
插入第12个元素 55 → 中序遍历: 10 20 25 30 35 40 45 50 55 60 70 80
插入第13个元素 65 → 中序遍历: 10 20 25 30 35 40 45 50 55 60 65 70 80
插入第14个元素 75 → 中序遍历: 10 20 25 30 35 40 45 50 55 60 65 70 75 80
插入第15个元素 85 → 中序遍历: 10 20 25 30 35 40 45 50 55 60 65 70 75 80 85
```

```
=====
第二步：删除元素，每次删除后输出 层序遍历
```

```
=====
删除第1个元素 20 → 层序遍历: 50 30 70 25 40 60 80 10 35 45 55 65 75 85
删除第2个元素 60 → 层序遍历: 50 30 70 25 40 65 80 10 35 45 55 75 85
删除第3个元素 30 → 层序遍历: 50 35 70 25 40 65 80 10 45 55 75 85
删除第4个元素 80 → 层序遍历: 50 35 70 25 40 65 85 10 45 55 75
```

```
任务完成！
```

插入和删除的时间以及空间复杂度：

$$O(\log n)$$

Difficulties encountered and solutions

删除双分支节点后失衡调整逻辑复杂。统一用右子树最小值替换，再递归删除最小值节点，最后按四种失衡情况调整。

关于Node的概念辨析不清，在插入和删除节点过程中有点乱。仔细阅读并理解代码，通过具体话题来完成理解。